FACULTAD DE LA ENERGÍA, LAS INDUSTRIAS Y LOS RECURSOS NATURALES NO RENOVABLES

QUINTO CICLO "A"

TEMA: Resumen U1 Sistemas Digitales

CARRERA:

Computación

DOCENTE:

Ing. Luis Darío Sinche Cueva

ASIGNATURA:

Sistemas Digitales

ESTUDIANTE:

Alex Francisco Sigcho Serrano

FECHA:

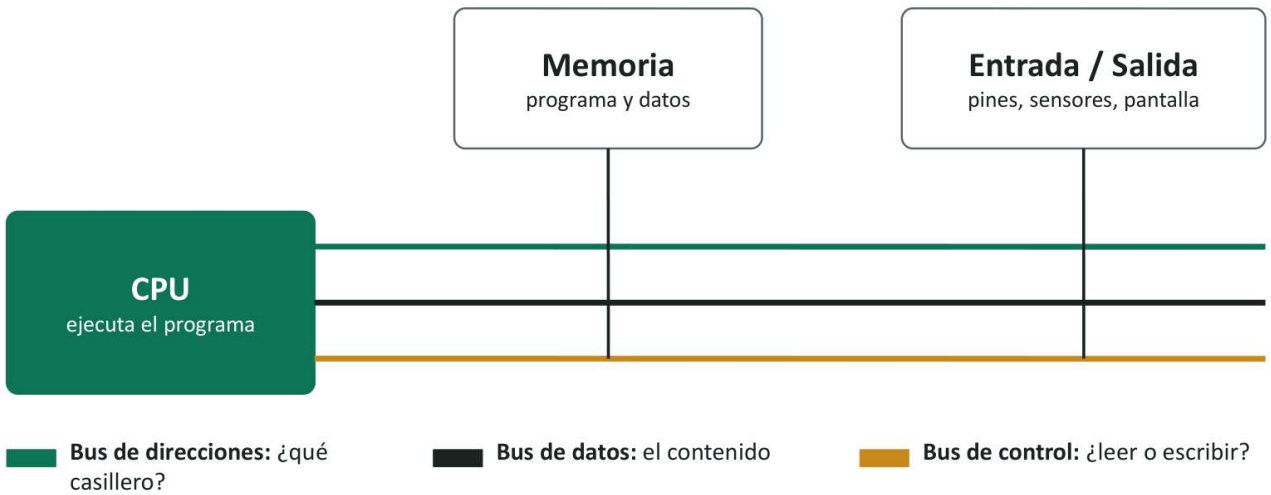
10 de noviembre del 2025

Sistemas digitales y ESP32

Resumen de las seis presentaciones de la unidad. Abarca la CPU, la memoria y el GPIO, hasta llegar a programas que reaccionan a pulsadores y alarmas.

Sistema embebido Cumple una función concreta. En una lavadora, lee los sensores y controla los actuadores según el ciclo.	**Tres buses** Direcciones → dónde. Datos → qué valor. Control → qué operación, como leer o escribir. Para guardar 0x20 en 0x25, la CPU selecciona 0x25, pone 0x20 en el bus de datos y ordena escribir.
Procesador y MCU El microprocesador aporta la CPU y suele necesitar otros chips. El microcontrolador (MCU) integra CPU, memorias y periféricos.	**Ciclo de instrucción** Buscar → decodificar → ejecutar. El reloj marca el ritmo del hardware. Una instrucción puede requerir varios ciclos.
Dentro de la CPU La unidad de control coordina las instrucciones. La ALU calcula y aplica lógica. Los registros guardan datos y el contador de programa señala la siguiente instrucción.	**Elección del chip** Se elige según memoria, pines, conectividad y consumo. Para una tarea simple sin Wi-Fi puede bastar un MCU pequeño.

CPU, memoria, E/S y buses



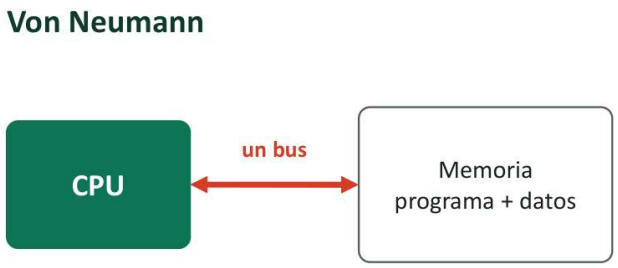
Memoria y periféricos colaboran mediante buses. Un periférico especializado, como un temporizador, puede trabajar mientras la CPU ejecuta otras instrucciones.

Arquitectura y memorias

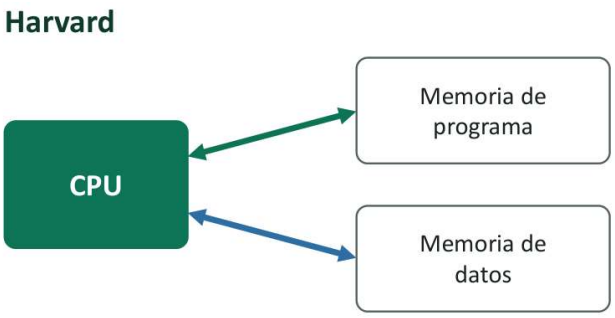
La arquitectura organiza el acceso a instrucciones y datos. El tipo de memoria determina qué permanece cuando se corta la alimentación.

Von Neumann En el modelo clásico, instrucciones y datos comparten memoria y canal de acceso. La CPU usa ese camino para ambos.	**Flash** Conserva el programa sin alimentación. Borrar y reescribir repetidamente desgasta sus celdas. Se usa para firmware y datos persistentes.
Harvard Separa los caminos de programa y datos. El ATmega328P ilustra esta idea al usar Flash para el programa y SRAM para datos de trabajo.	**SRAM** Guarda variables, pila y resultados temporales mientras el circuito está encendido. Su contenido se pierde al apagar.
Capacidad 1 byte = 8 bits. En unidades binarias, 1 KiB = 1024 B y 1 MiB = 1024 KiB. Las diapos usan KB y MB con ese sentido al hablar de memoria.	**EEPROM** En el AVR conserva datos pequeños como una calibración. El ESP32 puede guardar configuraciones en Flash mediante mecanismos como NVS.

Caminos de programa y datos



Instrucción y dato se traen por turnos, por el mismo bus.



Dos buses: la siguiente instrucción llega mientras se

Ejemplo: El código del termostato va en Flash, la temperatura del instante en SRAM y la calibración en memoria persistente. Evita guardar en Flash un valor que cambia en cada vuelta.

Bits hex y máscaras

Un bit representa 0 o 1. En una señal digital, HIGH y LOW representan niveles lógicos. Los voltajes que corresponden a cada nivel dependen del circuito.

Pesos del byte 128 · 64 · 32 · 16 · 8 · 4 · 2 · 1. Se suman los pesos que tienen un 1. Así, 0010 0000 vale 32 y 1000 0001 vale 129.	**Crear la máscara** 1U << 5 mueve el 1 cinco posiciones y produce 0010 0000. El desplazamiento señala una posición concreta dentro del registro.
Hexadecimal Cada grupo de 4 bits se escribe con un dígito de 0 a F. 0010 0000 = 0x20. 0xFF = 1111 1111 = 255. El bit 0 es el de la derecha.	**Cambiar un bit** Encender con reg |= m. Limpiar con reg &= ~m. Alternar con reg ^= m. Comprobar con (reg & m) != 0.
Lógica bit a bit AND (&) exige dos unos. OR (|) exige al menos uno. XOR (^) da uno si difieren. NOT (~) invierte los bits.	**Cuenta rápida** Con reg = 1010 1010 y m = 0000 0100, OR da 1010 1110. Solo se activa el bit 2.

Desplazamiento del bit 0 al bit 5



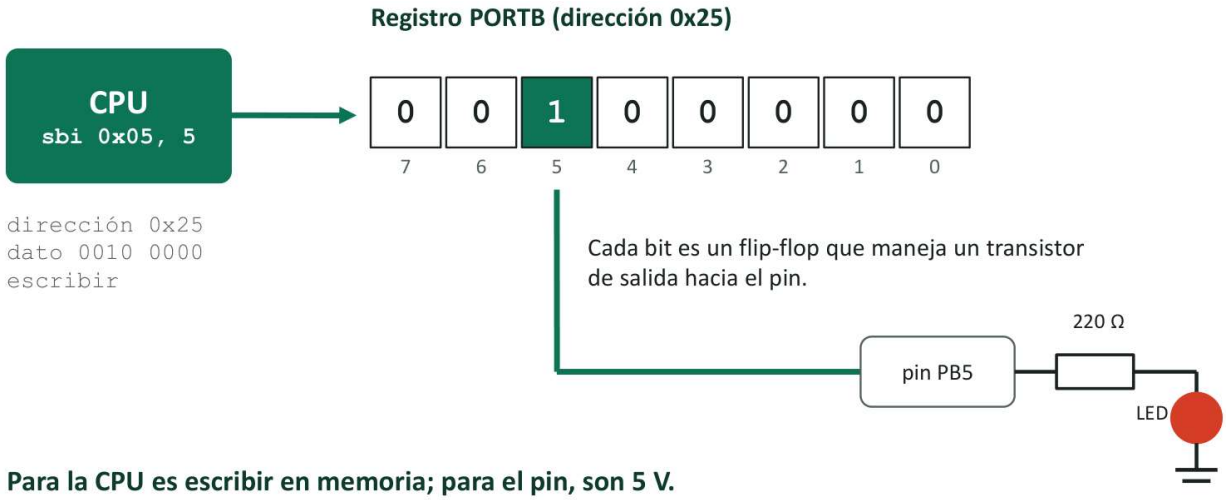
0000 1111 = $2^4 - 1 = 15$. El atajo funciona para los cuatro unos consecutivos que empiezan en el bit 0. Para unos separados, suma sus pesos.

Registros y entrada salida

La CPU controla periféricos leyendo o escribiendo direcciones asociadas a registros. En el AVR, cambiar un bit de PORTB puede modificar una salida física.

DDRB define el sentido DDRB5 = 1 configura PB5 como salida. DDRB5 = 0 lo configura como entrada.	**De C a instrucciones** PORTB |= (1 << 5) expresa la operación en C. El AVR dispone de instrucciones como LDI para cargar un inmediato y SBI para poner a 1 un bit de ciertos registros de E/S.
PORTB escribe o activa pull up En salida, PORTB5 = 1 pone PB5 en HIGH y 0 lo pone en LOW. En entrada, el bit en 1 activa la resistencia pull-up interna.	**RISC y CISC** Describen el estilo del juego de instrucciones. El AVR es RISC y su ancho de 8 bits describe cómo maneja datos en la CPU.
PINB lee el pin Se consulta PINB y se aplica una máscara para aislar PB5. En el Arduino Uno, D13 corresponde a PB5.	**Equivalencia en Arduino** pinMode(13, OUTPUT) configura la salida. digitalWrite(13, HIGH) la pone en alto. La función de Arduino oculta los detalles de los registros.

Del registro PORTB al LED



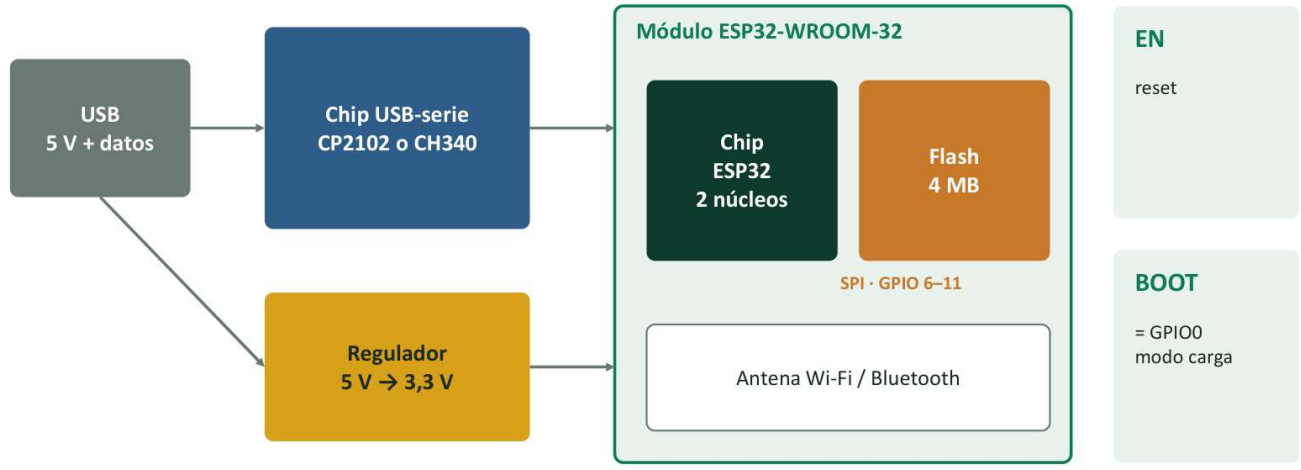
Ejemplo Con DDRB5 = 0 y PORTB5 = 0, PB5 queda como entrada sin pull-up. Un pulsador con pull-up permite obtener un nivel definido cuando está suelto.

Del AVR a la placa ESP32

La idea de controlar pines mediante registros se mantiene. El ESP32 amplía el ancho de palabra y permite asignar muchas señales de periféricos a distintos GPIO.

ATmega328P en Arduino Uno 8 bits, un núcleo a 16 MHz, 2 KB de SRAM y 32 KB de Flash interna. En el Uno la lógica trabaja a 5 V.	**Pin y periférico** El pin es el contacto físico. El periférico es el circuito interno, como UART, SPI o PWM. GPIO significa entrada/salida de propósito general.
ESP32 WROOM 32 32 bits, dos núcleos Xtensa LX6 de hasta 240 MHz y 520 KB de SRAM. El módulo de clase lleva 4 MB de Flash SPI y ofrece Wi-Fi y Bluetooth.	**GPIO Matrix e IO MUX** En el ESP32 permiten enrutar muchas señales internas hacia pines compatibles. La disponibilidad depende de cada GPIO y de la placa.
Chip módulo y placa El chip contiene la CPU y periféricos. El módulo añade Flash y antena. La DevKit añade USB-serie, regulador, botones EN y BOOT, y pines accesibles.	**Capas de abstracción** Tu código → Arduino-ESP32 → ESP-IDF → registros → hardware. Las funciones de alto nivel simplifican el uso. Los registros ofrecen control directo.

Partes de la DevKit V1



EN reinicia el chip. BOOT permite entrar al modo de carga al arrancar. El USB de la DevKit se comunica con el ESP32 mediante un puente USB-serie.

Registros GPIO del ESP32

Los registros separan la configuración del sentido, la escritura de salidas y la lectura de entradas.

Dirección y lectura GPIO_ENABLE_REG habilita salidas y GPIO_IN_REG lee entradas del banco GPIO 0–31. El banco superior cubre GPIO 32–39, con las limitaciones de cada pin.	Riesgo de concurrencia Leer, modificar y escribir puede perder un cambio si una ISR actualiza el mismo registro entre la lectura y la escritura. La última escritura puede sobrescribir la actualización.
Estado de salida GPIO_OUT_REG guarda el valor de salida del banco inferior. W1TS y W1TC permiten actuar sobre bits seleccionados mediante una sola escritura.	Ejemplo de encendido Si OUT = 0101 y se escribe 0010 en W1TS, OUT pasa a 0111. Si luego se escribe 0001 en W1TC, OUT queda 0110.
W1TS y W1TC En W1TS, un 1 en la máscara enciende el bit correspondiente. En W1TC, un 1 lo apaga. Un 0 en la máscara conserva ese bit.	Aplicación con un LED Configura GPIO16 como salida y usa una máscara 1UL << 16. Con los registros del banco inferior puedes encender y apagar ese bit.

Máscaras W1TS y W1TC



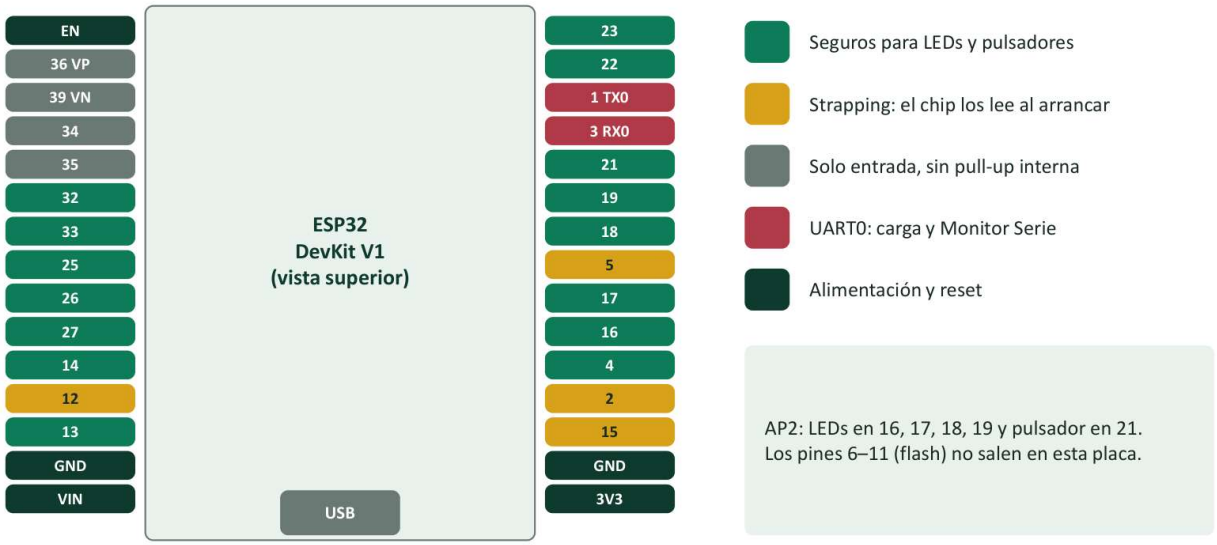
Al escribir W1TS o W1TC, la máscara indica qué bits tocar. La operación evita la lectura previa del registro de salida.

Pines seguros y arranque

El número GPIO identifica una señal. Para ubicar el contacto físico en la DevKit V1, consulta los rótulos y el mapa de la placa.

Pines de la práctica El juego de luces usa GPIO16, 17, 18 y 19 para LEDs y GPIO21 para el pulsador. GPIO34, 35, 36 y 39 son solo entrada y requieren resistencias externas si se usa pull-up.	**De reset a loop** La ROM lee los pines de arranque y carga el segundo bootloader desde Flash. Luego arranca la aplicación. En Arduino-ESP32, loopTask ejecuta setup() una vez y repite loop().
Pines reservados y de arranque GPIO6–11 se usan para la Flash. GPIO0, 2, 5, 12 y 15 intervienen en el arranque. GPIO1 y 3 se usan habitualmente para UART0.	**Superlazo** Una vuelta típica lee entradas, decide y actualiza salidas. Un bucle que gira sin esperar puede gastar energía al mantener la CPU activa.
Lógica de 3,3 V Usa GND común y niveles compatibles. Para un LED rojo de Vf ≈ 2,0 V y 330 Ω, I = (3,3 − 2,0)/330 ≈ 4 mA.	**Herramientas de clase** Arduino IDE 2.x con el core Arduino-ESP32 3.x. Si falla la carga, revisa cable de datos, puerto, placa seleccionada y el modo BOOT.

Mapa de pines de la DevKit V1 de 30 pines



Conexión del LED GPIO → resistencia → LED → GND. La resistencia limita la corriente. VIN y 3V3 son pines de alimentación y EN es de reinicio.

Tipos y desbordamiento

El tipo de dato define cuántos bits ocupa una variable y cómo se interpreta. El tamaño de int puede cambiar entre plataformas.

Tamaño explícito uint8_t cubre 0 a 255. int8_t cubre −128 a 127. uint16_t cubre 0 a 65 535. uint32_t cubre 0 a 4 294 967 295.	**Desbordamiento** Los tipos sin signo vuelven al inicio al superar el máximo. Un uint8_t pasa de 255 a 0. El desbordamiento aritmético de un int con signo tiene comportamiento indefinido.
AVR frente a ESP32 int ocupa 2 bytes en el AVR de clase y 4 en el ESP32. Usa tipos de stdint.h cuando importa el tamaño de una máscara, un dato o un tiempo.	**Constantes** constexpr uint8_t PIN_LED = 16 declara una constante con tipo conocida al compilar. #define hace sustitución de texto. const también declara un valor de solo lectura.
Promoción y literales ~a suele operar tras promover un uint8_t a int. (uint8_t)~a conserva los 8 bits bajos. Usa 1UL << 31 para una máscara sin signo del bit 31 en el ESP32.	**static y volatile** Una variable local static conserva su valor entre llamadas. volatile indica al compilador que haga los accesos previstos a un dato que puede cambiar fuera del flujo normal.

Vuelta de un contador de 8 bits



Ejemplo: for (uint8_t i = 0; i < 256; i++) no termina por esa condición. i vuelve a 0 antes de llegar a 256. Usa un tipo que pueda representar el límite.

Tiempo sin bloquear

Cuando un programa debe hacer varias tareas, conviene que cada vuelta de loop() pueda leer entradas y decidir si ya pasó el intervalo de cada tarea.

Qué hace delay delay(500) detiene el avance de la tarea que ejecuta loop() durante medio segundo. Un toque breve puede pasar entre dos lecturas del pulsador.	**Vuelta del contador** Con 8 bits, de 250 a 4 pasan 10 pasos. La resta sin signo conserva la diferencia al cruzar el cero si ha transcurrido menos que una vuelta completa. Con millis() de 32 bits, la vuelta tarda unos 49,7 días.
Qué hace millis millis() devuelve el tiempo transcurrido en milisegundos. Guardar ahora y el instante anterior permite saber cuánto pasó sin detener loop().	**Dos ritmos en paralelo** Guarda un instante anterior y un intervalo por LED. Cada vuelta evalúa ambas condiciones y permite seguir leyendo el botón.
Patrón de comparación Usa uint32_t para ambos tiempos. La condición es (ahora - anterior) >= intervalo. Al cumplirla, actualiza anterior y realiza la acción.	**Cuando usarlo** Es apropiado para tareas que pueden atenderse en la siguiente vuelta de loop(). Un timer de hardware es útil cuando se necesita un evento periódico independiente del avance del programa.

Lectura del botón con una espera bloqueante



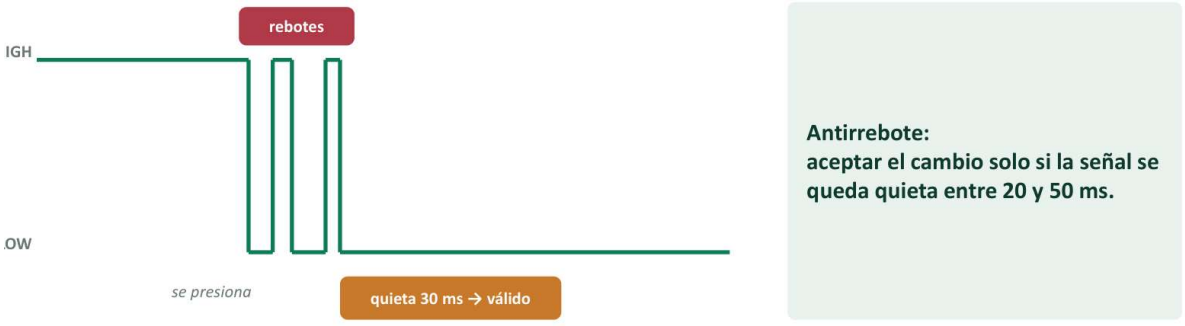
Regla de bolsillo: Guarda instantes y calcula diferencias. Comparar ahora con anterior + intervalo puede fallar cerca de la vuelta del contador.

Pulsadores y antirrebote

Para detectar una pulsación hace falta un nivel de reposo definido y un criterio para separar los rebotes de un cambio real.

Pull up Con INPUT_PULLUP y el botón entre GPIO21 y GND, el pin está en HIGH cuando está suelto y pasa a LOW al presionar.	**Por qué rebota** El contacto mecánico puede abrir y cerrar varias veces al cambiar de estado. Una pulsación física puede producir varios flancos eléctricos.
Entrada flotante Si el pin queda sin una referencia al soltar el botón, la lectura puede variar. Una resistencia pull-up mantiene el estado alto en reposo.	**Filtro por estabilidad** Cada cambio en la lectura reinicia el tiempo de espera. Si la lectura permanece igual durante 30 ms, se acepta como nuevo estado estable.
Nivel y flanco El nivel responde si el botón está presionado ahora. El flanco indica que cambió. Con pull-up, la pulsación corresponde al flanco de bajada HIGH → LOW.	**Contar una sola vez** Al aceptar un nuevo estado estable LOW se suma una pulsación. Si el botón se mantiene presionado, el estado se conserva y el contador solo suma de nuevo después de soltar y volver a presionar.

Señal de un pulsador con rebote



lecturaAnterior detecta cambios, tUltimoCambio mide cuánto duró el nuevo nivel y estadoEstable guarda el último nivel confirmado.

Temporizador reloj y prescaler

Un temporizador de hardware mide el tiempo contando pulsos de una fuente de reloj. La CPU configura el temporizador y luego el periférico cuenta por su cuenta.

Contador Es un registro que cambia con cada tick. Puede empezar en cero. La CPU configura el arranque, la dirección de conteo y el momento de generar una alarma.	Alarma Es el valor de comparación. Al alcanzarlo, el temporizador genera el evento. Si se cuenta desde cero, tiempo = alarma / f_tick.
Reloj y divisor En los ejemplos se parte de un reloj APB de 80 MHz. Un prescaler N deja avanzar el contador una vez por cada N pulsos del reloj	Ejemplo de 500 ms A 1 MHz, 500 ms = 0,5 s. Alarma = 0,5 × 1 000 000 = 500 000 ticks.
Frecuencia y resolución Frecuencia de tick = f_reloj / N. Resolución = 1 / f_tick. Si f_tick = 1 MHz, cada tick dura 1 µs.	Ejemplo inverso Con f_tick = 10 kHz y alarma = 2500, cada tick dura 100 µs. El evento ocurre tras 0,25 s, es decir, 250 ms.

División del reloj y frecuencia de tick



Prescaler: divisor que deja pasar al contador solo 1 de cada N pulsos del reloj. El contador avanza N veces más lento.

Tick del timer: cada pulso que sale del prescaler; es cuando el contador suma 1.

Un divisor más grande produce ticks más largos. Permite medir intervalos mayores antes de llenar el contador, con una resolución más gruesa.

Alarmas autorrecarga y API

Una vez fijada la frecuencia del contador, la alarma determina cuándo se produce el evento. La autorrecarga permite repetirlo de forma periódica.

Alarma con autorrecarga Al alcanzar el umbral se genera el evento y el contador vuelve al valor de recarga. Si vuelve a cero, el siguiente intervalo dura lo mismo.	Comparación con el AVR El ATmega328P tiene tres temporizadores. Timer0 y Timer2 son de 8 bits y Timer1 es de 16 bits. Un ancho mayor permite contar más ticks antes de desbordarse.
Desbordamiento natural El contador ascendente vuelve a cero al pasar su máximo $2^n - 1$. En 8 bits el máximo es 255. La autorrecarga, en cambio, responde a un umbral configurado.	Core y API El core añade el soporte del ESP32 al Arduino IDE. Su API configura los registros del periférico. Las firmas de temporizador cambiaron entre Arduino-ESP32 2.x y 3.x.
Timers del ESP32 clásico El ESP32-WROOM-32 tiene cuatro temporizadores de propósito general, repartidos en dos grupos. Cada uno dispone de un contador de 64 bits y un prescaler de 16 bits.	Cuando llega la alarma Se puede asociar una función de atención o callback. La ISR marca una bandera y regresa. loop() atiende el aviso y hace tareas como imprimir.

Ciclo de alarma con autorrecarga



Autorrecarga (auto-reload): al alcanzar la alarma, el contador vuelve solo a 0. Así el aviso se repite a intervalos iguales sin que el programa haga nada.

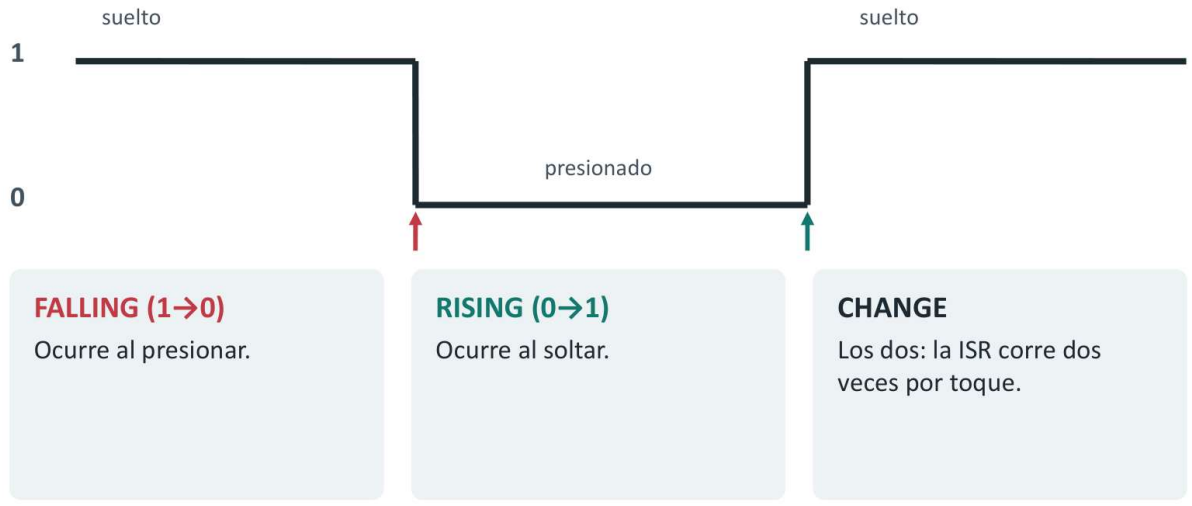
Con millis() el programa consulta el tiempo en cada vuelta. Un temporizador configurado con alarma puede generar el evento, aunque el superlazo esté ocupado.

Interrupciones externas y flancos

Una interrupción externa empieza con un evento en un GPIO. El periférico detecta el cambio y avisa a la CPU, que ejecuta la ISR antes de retomar el flujo interrumpido.

Sondeo loop() consulta el pin periódicamente. Si el botón se presiona y se suelta entre lecturas, el programa puede perder el toque.	**Registro de la función** En setup(), configura pinMode(21, INPUT_PULLUP) y luego llama a attachInterrupt(digitalPinToInterrupt(21), isrBoton, FALLING).
Interrupción El GPIO vigila el pin. La CPU guarda el contexto necesario, atiende el evento en la ISR y luego continúa su ejecución.	**Qué GPIO usar** GPIO21 es el de la práctica. Los GPIO34–39 disponibles son solo entrada y no tienen pull-up interna. Se puede usar una resistencia externa para fijar su reposo.
Tres modos RISING detecta LOW → HIGH. FALLING detecta HIGH → LOW. CHANGE detecta ambos. Un pulsador con pull-up produce FALLING al presionar y RISING al soltar.	**Origen del aviso** La interrupción de timer proviene de un contador interno. La interrupción externa proviene del pin. Ambas pueden usar la misma separación entre aviso rápido y trabajo posterior.

Flancos de una pulsación con pull-up



ISR IRAM y datos compartidos

La ISR es la rutina que atiende la interrupción. Su trabajo debe ser breve para que el sistema pueda seguir atendiendo el programa y otros eventos.

Qué poner en la ISR Marca una bandera o registra un dato breve con operaciones adecuadas para interrupción. Deja las impresiones, esperas y tareas pesadas para loop().	**IRAM y DRAM** IRAM aloja instrucciones. DRAM aloja datos. IRAM_ATTR coloca la función en IRAM. Si la ISR debe ejecutarse con la caché deshabilitada, el código que llama y los datos que toca también deben ser accesibles.
Watchdog Vigila que el sistema pueda seguir atendiendo sus tareas. Si una ISR ocupa la CPU demasiado tiempo, el watchdog de interrupciones puede dispararse y provocar un reinicio.	**volatile** Usa volatile para la bandera compartida por ISR y loop(). Así el compilador conserva sus lecturas y escrituras. Un dato privado de la ISR, como tUltimo en la práctica, tiene otro uso.
Flash y caché El programa normalmente se guarda en Flash y la caché acelera su lectura. Durante ciertas operaciones sobre la Flash, el acceso por caché puede quedar temporalmente desabilitado.	**Atomicidad** volatile no convierte una secuencia de leer, modificar y escribir en una operación indivisible. Para contadores compartidos o para leer y limpiar sin perder avisos, usa sincronización adecuada.

Instrucciones en IRAM y datos en DRAM

IRAM · Instruction RAM

Guarda instrucciones: código que la CPU ejecuta.

Aquí pondremos la ISR.

DRAM · Data RAM

Guarda datos: por ejemplo, las variables globales.

Aquí vive la variable alerta.

Botón alarma y repaso final

En la práctica, el botón de GPIO21 activa una alerta que loop() usa para cambiar un LED en GPIO19. El recorrido une GPIO, flanco, ISR, tiempo y estado.

Montaje y disparo GPIO21 se configura con INPUT_PULLUP. El botón cierra hacia GND y se usa FALLING para detectar la pulsación.	Bandera o contador Un bool indica que hubo al menos un aviso. Si llegan tres flancos válidos antes de atenderlo, pueden quedar en un solo aviso. Un contador permite registrar cuántos llegaron, con sincronización al compartirlo.
Filtro por intervalo La diapositiva de clase usa millis() y acepta un flanco si han pasado al menos 50 ms desde el último aceptado. El filtro por intervalo difiere del filtro por estabilidad de la semana 2.	Elegir el mecanismo Sondeo para revisar entradas en cada vuelta. millis() para planificar sin bloquear. Timer para alarmas periódicas por hardware. Interrupción GPIO para reaccionar a un flanco externo.
ISR y bucle principal La ISR actualiza el instante y levanta alerta. loop() recoge el aviso, cambia el LED y puede imprimir. Con tUltimo = 0 se ignora un flanco ocurrido antes de los primeros 50 ms.	Repaso de unidades A 1 MHz, cada tick dura 1 µs. Para una alarma cada 20 ms desde cero, configura 20 000 ticks. Con autorrecarga se repite el intervalo.

Recorrido desde el pulsador hasta loop()

